

免patch激活软件教程

——以aida64 business/extreme等为例

Abstract

本文介绍了一种破解软件的思路，利用该思路，可以不改目标软件任何文件一个字节实现激活，并且使得软件可以在一定时间内自由升级保持激活状态。

背景和基本思路

论坛上能看到很多破解教程，也看到有人为了减少修改，实现了RSA公钥只改一个字节的机制。然而，改一个字节也是改，改了就会影响数字签名，也可能导致杀软报错。有没有办法，一个字节也不改呢？当然，办法是有的。

早些天我在本论坛就放了一个某软件一字节也不改的激活方案，用过的朋友都说好。今天，我把这种方法拿出来，给大家讲解一下，对于初学者是很好的学习材料，对于高手，也算提供一种参考思路。

当然，一个字节也不改，又要修改目标软件的逻辑，方法就不多了。大致有几种：

1. 利用驱动在内核级修改目标软件的逻辑
2. 利用调试API强制介入修改目标软件
3. 利用DLL劫持做目标软件的进程内Patch。

这几种方法我都试过，但在今天方法1可以直接淘汰了，因为缺点很明显，太难放驱动并且可能导致系统随时挂掉。方法2对于某些防dll劫持很严的软件来说是必要的。但市面上大多软件都没有做防dll劫持。dll劫持是一种很简单很方便的方案，适合大多数场景，本文就介绍这个。

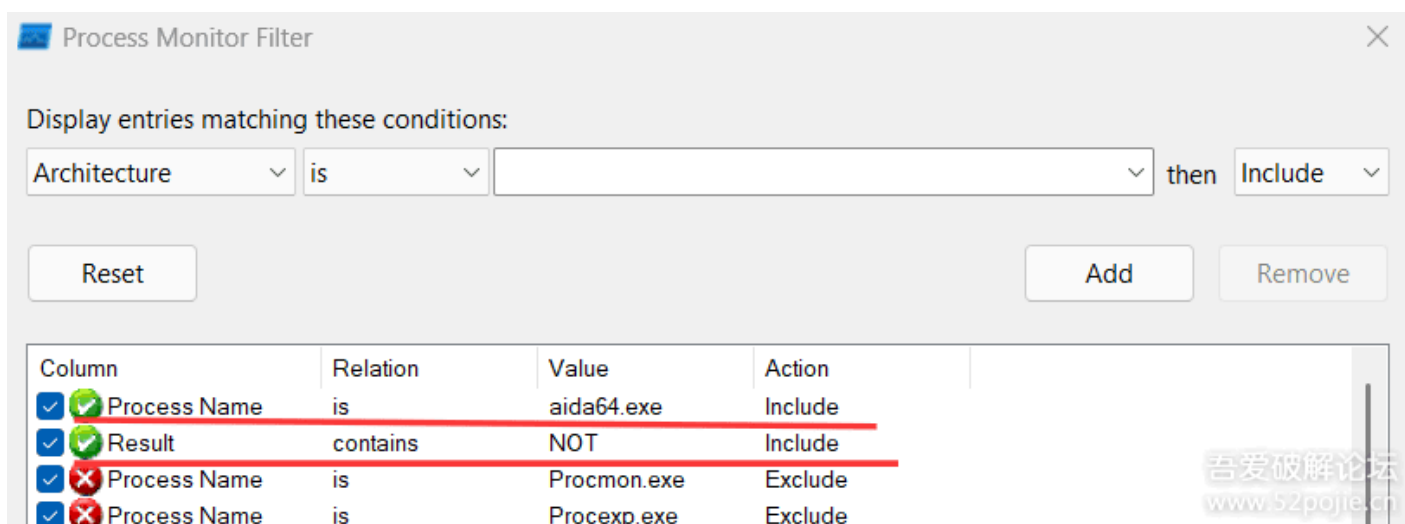
注意：本文要求读者有一定的编程能力，对windows操作和破解有一些基本的了解，对C++有一定的理解能力，能使用Visual Studio编译出产品。本文用到的代码使用了C++20标准。读者需要能大致理解。当然，如果不理解，可以问AI。

DLL劫持简介

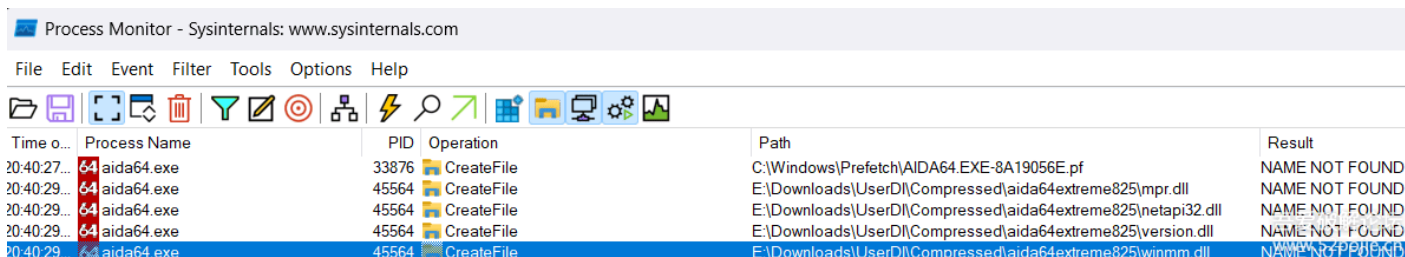
Windows 进程中exe要依赖大量的系统API和第三方API。当exe启动时，它会加载一些dll，而这些dll又会加载其它的dll。Windows维护了一个已知DLL列表，对于列表内的，它会直接去系统目录查找dll，而对于不在这个列表中的，它会首先尝试在exe所在目录加载。这样，当exe用到了xxx.dll时，如果我们在exe目录放一个假的xxx.dll，系统就会优先加载它，我们在假的xxx.dll中可以做一些事情，然后把API调用转到真的xxx.dll，就可以完成目标软件的修改了。

选择要劫持的DLL

我们以aida64为例，从地址 <https://download2.aida64.com/aida64extreme825.zip> 下载一个aida64 extreme（当然你把extreme改成business也行，对于本文接下来的内容不影响）。解开待用。打开 procmon64 (<https://learn.microsoft.com/en-us/sysinternals/downloads/procmon>)，按图填入Filter（填时记得点Add），点击OK。



双击启动 aida64.exe，看看结果里，有很多显示DLL找不到的条目。



可以看出，运行aida64时，它会尝试加载 mpr.dll,netapi32.dll,version.dll,winmm.dll等这些dll，由于这些dll不在已知DLL名单内，会首先尝试从exe所在目录加载。我以winmm.dll为例，我只需要在aida64目录放一个假的winmm.dll，它就会被加载，然后它调用真的winmm，这一切不就OK了？

假 winmm.dll 代码详解

为了制造一个假的winmm.dll，我们可以用vs2022，随便创建一个新的dll工程，编一个dll改名成winmm就行。

注意，我这里不再从0开始介绍一步步创建工程和加入代码，我把成品代码放在了：

<https://github.com/swigger/aida64-dll-inject-example>

我们直接用它来讲解。

我们知道dll加载后会调用DllMain函数，在dllmain.cpp中的该函数里，如果判断是进程启动加载，我首先调用了 ui::start，这里是为了后续自动填入序列号。然后，我们判定主exe的状态，是不是upx版本，如果是，则hook 导入表中的 GetProcAddress，等upx在内存中解压完成后再调用我们的hook逻辑。否则的话用 QueueUserApc 启动一个异步回调，在dll全加载完，exe开始运行前，调用我们的hookFunc。

这个逻辑是因为默认的aida64.exe是upx压缩过的，我们要求一字节不改，当然要支持主exe是upx压缩的情况。不过在调试期间，我们可能已经用upx -d解出原始的aida64了，所以也要支持不是upx的情况。

工程中有一个winmm.def，导出了winmm的常见函数。这些是假winmm能工作的关键，我们的假winmm必须能支持真正的函数，支持方法就是跳到真函数去。我用脚本生成了winmm.def.asm，对于每一个假导出，它会跳转到一个函数，默认是跳到加载真导出地址再跳转到真地址的函数，在第一次找到真地址后，以后就只跳到真地址不再请求加载地址。

命名空间ui中的函数用于在弹出请求输入验证码那个窗口时，在内存中生成一个序列号，填上去。它首先一次性启动一个当前线程的CBT钩子，观察窗口变化的事件，当一个窗口第一次被激活时，判定是不是请求输入验证码的对话框，如果是，则延时10ms后，找到输入序列号的文本框，在内存中生成一个序列号，填好。

注意：为了尊重本论坛不放成品破解软件的规定，我这个工程里生成的序列号是假的，通不过验证，大家可以找到真实的算法，替换一下，就可以看到想要的结果。

在 hookFunc 中，我们只做一件事，找到 LStrFromPChar 函数，然后 hook 它。

为什么要这么做？

因为 aida64 有网络验证，如果你输入假的key，当场是通过了，但时不时的它就跳出一个窗口，说你的key是假的，你说气人不气人。经过调试验证，它是在网络返回的xml中，识别特定信息然后弹框的。这个信息就是服务器在xml中放置的 <nod> <nod1> 等信息。而服务器的xml会首先经过LStrFromPChar函数转成delphi可识别的字符串再进行后续操作的，所以只需要hook这个函数，把nodXX这些节点去掉，不就解决网验了？从此不就可以放心点在线升级？

要hook LStrFromPChar，有两个事要做：第一是找到它，第二是hook它到我们的自有函数。

要找到它并不难，因为aida64是用delphi做的，而这个LStrFromPChar又是delphi的内置函数，而delphi又不会有二进制级的变态优化，这就意味着LStrFromPChar函数的二进制码，在忽略重定位的情况下，是不会变的！因此，我们直接hardcode其前16字节，暴力在code段中查找一下，不就找到了？

然后是hook它。有一些现成的第三方hook可以用。比如微软的detours，比较知名的还有mini hook。我这里用了一个我自己手撸的直接hook，它采用破坏rax的方法来实现hook，适合在函数开始处进行hook。需要修改5或12字节。它虽然还有一些潜在的bug，也没有经过充分的测试，但胜在代码非常简短，并且很容易读懂和修改。暂时未实现unhook方法（很容易实现）。我在原代码中打包了，一并献给大家。

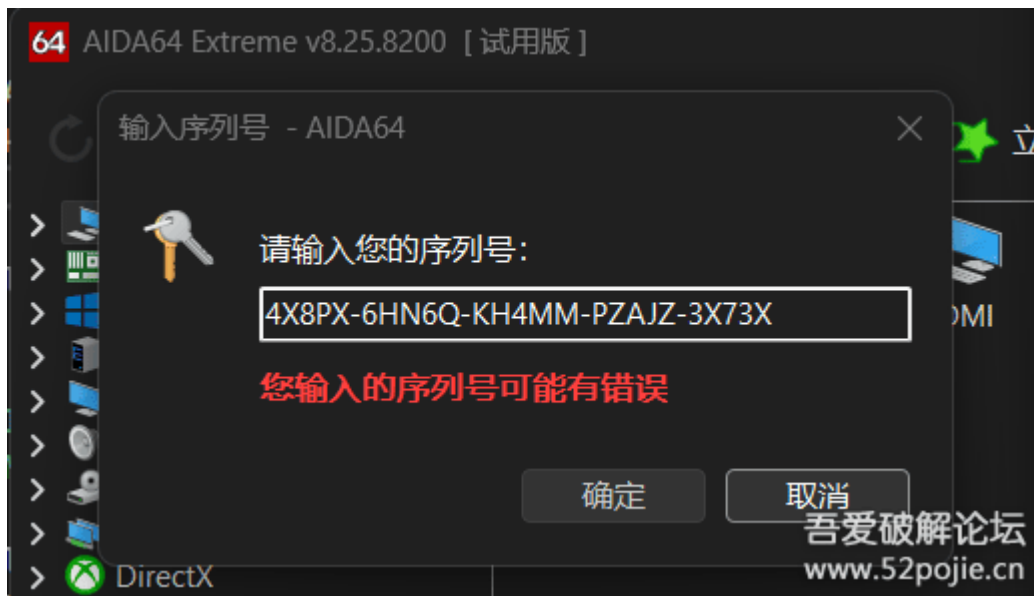
```
direct_hook(funcs[0], hook_LStrFromPChar, (void*)&trampo_LStrFromPChar);
```

这一句下来，我们把找到的 LStrFromPChar 函数，也就是funcs[0]，强制让它跳转到我们的实现函数 hook_LStrFromPChar，在这个函数中，把nodXX给去掉，然后调用原始函数。

这就解决了网验的问题。

使用方法

编译出 dll 后，把它拷贝到 aida64 所在目录，然后改名成 winmm.dll，然后打开 aida64，点击输入验证码，可以看到验证码自己填好啦。



注意：直接用本工程代码填的是假的，请大家自行修改keygen.cpp修正。

然后就注册成功了，再点击在线升级看看，静悄悄的，没有任何报假的弹窗！

NetUpdate - AIDA64

 升级  关闭

 正在检查更新

 正在连接到 Internet... ☒ 完成

 正在开始下载进程... ☒ 完成

 正在连接更新服务器... ☒ 完成

 正在下载更新信息... ☒ 完成

 可用的更新

 AIDA64 Stable 8.25.8200 (旧) ☒ 完成

☒ 在线更新完成

☒ AIDA64 已经是最新版本 ☒ 完成

 完成

吾爱破解论坛
www.52pojie.cn